PEMROGRAMAN BERORIENTASI OBJEK 2026

UAS: Ujian Akhir Semester

Persiapan UAS dan presentasi proyek akhir: cakupan, strategi, latihan soal, dan pembahasan

PERTEMUAN

16

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Memetakan materi Pertemuan 09—15 beserta hubungannya dengan fondasi Pertemuan 02—07.
2. Memprediksi *output* kode yang memakai polimorfisme, kelas abstrak, *exception*, dan koleksi, baik di Python maupun Java.
3. Menganalisis sebuah desain dengan prinsip SOLID lalu memilih *design pattern* yang tepat.
4. Merancang dan membuat program PBO lengkap: kelas abstrak, polimorfisme, *exception* buatan sendiri, dan koleksi.
5. Melakukan *refactoring* pada kode yang berantakan tanpa mengubah perilakunya, lalu membuktikannya.
6. Mempresentasikan proyek akhir secara runtut dan menjawab pertanyaan penguji dengan yakin.

Agenda Hari Ini

1. Cakupan, kisi-kisi, dan bentuk UAS
2. Peta konsep Pertemuan 02—15
3. Strategi belajar dan mengerjakan soal
4. Contoh latihan soal dan pembahasan
5. Jebakan favorit soal ujian
6. Presentasi proyek akhir

BAGIAN 1

1

Cakupan dan Bentuk UAS

Cakupan utama Pertemuan 09—15, dengan fondasi Pertemuan 02—07 yang tetap dipakai.

Cakupan Materi dan Kisi-Kisi

Pertemuan	Topik	Contoh kemampuan yang diuji	Latihan
09	Polimorfisme	memprediksi hasil *method* yang di-*override*; *overloading* vs *overriding*	A1, A2
10	*Abstract class* dan *interface*	aturan kelas abstrak; memilih kelas abstrak atau *interface*	A3, A4, B2
11	*Exception handling*	alur `try`/`finally`; urutan `except`/`catch`; *checked* vs *unchecked*	A5—A7, B3, C1
12	Koleksi dan *generics*	pengurutan dua kriteria; kesamaan objek; memilih koleksi	A8—A10, C1
13—14	SOLID dan *design pattern*	menemukan pelanggaran SOLID; memilih pola desain; *refactoring*	A11—A14, B4, B5, C2
15	Proyek terintegrasi	menempatkan aturan bisnis pada lapisan yang tepat	A15

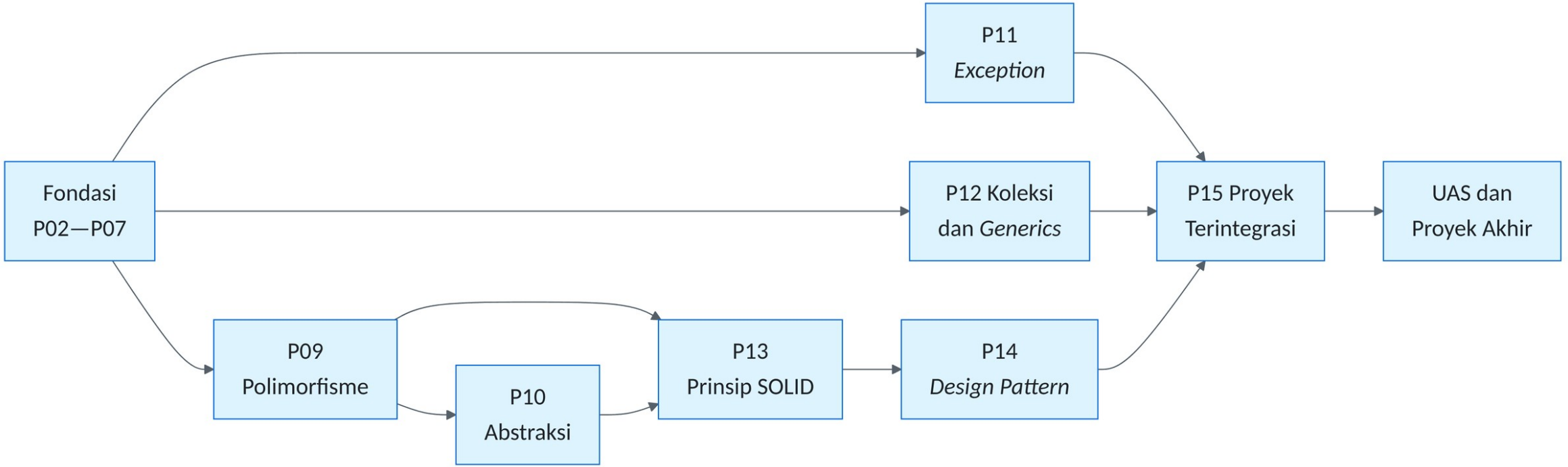
Fondasi Pertemuan 02—07 tetap dibutuhkan hampir di semua soal. Soal B1 menguji empat pilar PBO serta *is-a* dan *has-a*.

Bentuk Ujian yang Umum dan Latihannya

Bentuk	Yang biasanya diuji	Di latihan soal
Teori: pilihan ganda	konsep dan membaca kode: menebak *output*, mencari letak kesalahan	Bagian A: 15 soal, saran 30 menit
Teori: esai / analisis desain	menjelaskan, membandingkan, menganalisis SOLID, memilih *design pattern*	Bagian B: 5 soal, saran 60 menit
Praktik	membuat program PBO dari kebutuhan, atau *refactoring* kode yang ada	Bagian C: 2 soal, saran 150 menit

Bentuk, jumlah soal, bobot, durasi, dan aturan ujian yang sesungguhnya mengikuti pengumuman dosen.

Peta Konsep: dari Fondasi ke Proyek Akhir



Panah berarti “menjadi dasar bagi”. Kesulitan di satu kotak? Mundurlah ke kotak yang menunjuk ke sana.

BAGIAN 2

2

Strategi Belajar dan Mengerjakan

Kamu tidak perlu belajar semuanya dalam satu malam.

Rencana Belajar Tujuh Hari

Hari	Fokus	Kegiatan
H-7	P09—P10	baca ringkasan, jalankan ulang studi kasus P09 dan P10, tebak *output* dulu
H-6	P11	ubah salinan contoh `try/except` P11, tebak alurnya; buat *exception* sendiri
H-5	P12	latih `sorted(key=...)`/`Comparator` dua kriteria; kapan memakai `dict`/`Map`
H-4	P13—P14	gambar ulang diagram KedaiKita sebelum dan sesudah *refactor* dari ingatan
H-3, H-2	Simulasi	Bagian A dan B dengan *timer*, lalu Bagian C **tanpa** melihat solusi
H-1	Pemantapan	ulangi soal yang salah, isi *checklist*, siapkan proyek akhir, **tidur cukup**

Tips Mengerjakan Soal

Bentuk soal	*Tips*
Pilihan ganda membaca kode	baca dua kali (struktur, lalu alur); waspadai pilihan yang “hampir benar”
Esai dan analisis desain	**definisi → contoh → alasan**; tunjuk bukti di kode; sebut kekurangan pola
Praktik	**rancang dulu** 5—10 menit, tulis bertahap, jalankan setiap selesai satu tahap

Simpan pekerjaanmu secara berkala dan sisakan 10 menit terakhir untuk merapikan nama dan komentar.

BAGIAN 3

3

Contoh Latihan Soal

Tiga soal pilihan ganda dan potongan solusi soal praktik dari `latihan-soal.md`.

Kuis Cepat

Soal A8. Perhatikan kode berikut (versi Java memakai `Comparator.comparing(...).reversed().thenComparing(...)`). *Output* program adalah …

```
class Peserta:
    def __init__(self, nama, skor):
        self.nama = nama
        self.skor = skor
daftar = [Peserta("Dimas", 80), Peserta("Ayu", 92),
          Peserta("Citra", 80), Peserta("Bima", 75)]
urut = sorted(daftar, key=lambda p: (-p.skor, p.nama))
print(", ".join(p.nama for p in urut))
```

A. `Ayu, Dimas, Citra, Bima`

B. `Bima, Citra, Dimas, Ayu`

C. `Ayu, Citra, Dimas, Bima`

D. `Ayu, Bima, Citra, Dimas`

Kuis Cepat: Jawaban

Soal A8. Perhatikan kode berikut (versi Java memakai `Comparator.comparing(...).reversed().thenComparing(...)`). *Output* program adalah …

A `Ayu, Dimas, Citra, Bima`

B `Bima, Citra, Dimas, Ayu`

✓ `Ayu, Citra, Dimas, Bima`

D `Ayu, Bima, Citra, Dimas`

Jawaban: C

Kunci `(-p.skor, p.nama)` berarti skor dari besar ke kecil; jika skor sama, nama A—Z. Ayu (92) di depan, lalu Citra dan Dimas (80) menurut nama, dan Bima (75) di belakang. A melupakan kriteria kedua, B adalah urutan skor naik, dan D urutan nama saja.

Kuis Cepat

Soal A6. `StokTidakCukupError` (Java: `StokTidakCukupException`) adalah kelas anak dari `KedaiError` (Java: `KedaiException`). Mengapa blok `except`/`catch` untuk kelas anak itu harus ditulis **sebelum** blok untuk induknya?

A Hanya blok **pertama** yang cocok yang dijalankan; jika induk lebih dulu, blok anak tak tercapai

B Agar program berjalan lebih cepat karena *exception* kelas anak lebih ringan

C Karena *exception* kelas anak tidak bisa ditangkap oleh blok milik induknya

D Karena Python dan Java mewajibkan blok `except`/`catch` diurutkan menurut abjad nama kelasnya

Kuis Cepat: Jawaban

Soal A6. `StokTidakCukupError` (Java: `StokTidakCukupException`) adalah kelas anak dari `KedaiError` (Java: `KedaiException`). Mengapa blok `except`/`catch` untuk kelas anak itu harus ditulis **sebelum** blok untuk induknya?

✓ Hanya blok **pertama** yang cocok yang dijalankan; jika induk lebih dulu, blok anak tak tercapai

B Agar program berjalan lebih cepat karena *exception* kelas anak lebih ringan

C Karena *exception* kelas anak tidak bisa ditangkap oleh blok milik induknya

D Karena Python dan Java mewajibkan blok `except`/`catch` diurutkan menurut abjad nama kelasnya

Jawaban: A

Blok `except`/`catch` diperiksa dari atas, dan hanya blok **pertama** yang cocok yang dijalankan. `StokTidakCukupError` juga merupakan `KedaiError`, jadi blok induk yang ditulis lebih dulu menyerobot semuanya. Python diam saja, sedangkan Java menolaknya saat kompilasi. C justru terbalik: blok induk **bisa** menangkap *exception* anak.

Kuis Cepat

Soal A14. Aplikasi ojek *online* punya aturan tarif normal, jam sibuk, dan promo. Aturan dipilih saat aplikasi berjalan, dan aturan baru sering ditambahkan tanpa boleh mengubah kelas `Perjalanan`. Pola yang paling cocok adalah …

A *Strategy*

B *Singleton*

C *Observer*

D *Factory*

Kuis Cepat: Jawaban

Soal A14. Aplikasi ojek *online* punya aturan tarif normal, jam sibuk, dan promo. Aturan dipilih saat aplikasi berjalan, dan aturan baru sering ditambahkan tanpa boleh mengubah kelas `Perjalanan`. Pola yang paling cocok adalah ...

- ✓ *Strategy*
- B *Singleton*
- C *Observer*
- D *Factory*

Jawaban: A

Kebutuhannya adalah **menukar aturan perhitungan** saat program berjalan dan menambah aturan baru tanpa mengubah `Perjalanan`. Itulah *Strategy*: setiap aturan tarif menjadi satu kelas dengan *interface* yang sama. *Singleton* menjamin satu objek, *Observer* memberi tahu banyak pihak, dan *Factory* membantu membuat objek.

Solusi Soal C1: Kelas Abstrak `Tiket`

Satu induk abstrak dengan tiga kelas anak. `TiketVIP` menambahkan Rp25.000 ke harga dasar.

Python python/solusi/soal_c1_tiket_bioskop.py

```python
class Tiket(ABC):
    # ... (dilewati)
    @abstractmethod
    def hitung_harga(self) -> int:
        # ... (dilewati)
# ... (dilewati)
class TiketVIP(Tiket):
    TAMBAHAN = 25000
    # ... (dilewati)
    def hitung_harga(self) -> int:
        return self.harga_dasar + TiketVIP.TAMBAHAN
```

Java java/solusi/SoalC1TiketBioskop.java

```java
abstract class Tiket {
    // ... (dilewati)
    public abstract int hitungHarga();
    // ... (dilewati)
}
// ... (dilewati)
class TiketVIP extends Tiket {
    static final int TAMBAHAN = 25000;
    // ... (dilewati)
    @Override
    public int hitungHarga() {
        return getHargaDasar() + TAMBAHAN;
    }
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Tipe variabelnya induk, jadi yang jalan *method* induk.”	Yang jalan versi milik **objek sebenarnya** (*dynamic dispatch*), juga saat dipanggil dari dalam *method* induk.
“`finally` tidak jalan kalau sudah `return`.”	`finally` **tetap** jalan. Nilai kembaliannya ditahan dulu, `finally` dijalankan, baru nilai itu dikembalikan.
“Cukup meng-*override* `equals` saja.”	`equals` dan `hashCode` (atau `__eq__` dan `__hash__`) harus selalu berpasangan.

BAGIAN 4

4

Presentasi Proyek Akhir

Aplikasinya berjalan, desainnya masuk akal, dan kamu memahami kodemu sendiri.

Alur Demo 10—15 Menit (Saran)

Menit	Bagian	Yang ditunjukkan
0—1	Pembukaan	nama, tema, dan masalah yang diselesaikan aplikasimu (satu kalimat)
1—3	Rancangan	diagram kelas UML: pewarisan, kelas abstrak/*interface*, relasi
3—8	Demo aplikasi	alur utama, lalu 1—2 skenario gagal yang ditangani dengan rapi
8—11	Tur kode	bagian kode yang memenuhi setiap kebutuhan minimum
11—13	*Unit test*	jalankan semua *test* (minimal 5) di depan penguji
13—15	Refleksi	kesulitan terbesar, keputusan desain, rencana pengembangan, bantuan

Durasi, urutan, dan bobot presentasi yang pasti mengikuti ketentuan proyek akhir dan pengumuman dosen.

Pertanyaan yang Sering Diajukan Penguji

Pertanyaan	Yang ingin diketahui
“Kalau saya menambah jenis X baru, kelas mana yang berubah?”	OCP dan polimorfisme
“Mengapa memakai kelas abstrak, bukan *interface* (atau sebaliknya)?”	kemampuan memilih abstraksi
“*Exception* apa yang kamu buat sendiri? Mengapa tidak pakai yang bawaan?”	paham manfaat *exception* khusus
“Pola desain apa yang kamu pakai? Apa kekurangannya?”	paham pola, bukan hanya menyebut nama
“Jelaskan baris ini.” (penguji menunjuk kode acak)	keaslian pekerjaan
“Bantuan apa saja yang kamu pakai, termasuk AI?”	integritas akademik

Kamu harus bisa menjelaskan **setiap baris** kode proyekmu. Cantumkan semua bantuan yang kamu pakai di README.

Ringkasan

- Cakupan utama UAS adalah Pertemuan 09—15, dengan fondasi Pertemuan 02—07 yang tetap dipakai.
- Bentuk umum: pilihan ganda, esai atau analisis desain, dan praktik; format resmi dari dosen.
- Belajar bertahap tujuh hari: tebak dulu, jalankan kemudian, dan simulasikan ujian dengan *timer*.
- Pilihan ganda: baca kode dua kali. Esai: definisi → contoh → alasan. Praktik: rancang dulu.
- Waspadai jebakan: *dynamic dispatch*, `finally`, urutan `except`, serta `equals` dan `hashCode`.
- Presentasi: aplikasi berjalan, desain masuk akal, dan kamu memahami setiap baris kodemu.

Langkah Berikutnya

- Kerjakan `latihan-soal.md` sebagai simulasi, lalu cocokkan di `pembahasan.md`.
- Pelajari solusi praktik di jalurmu, lalu buktikan *refactor*-mu dengan `diff` atau `fc`.
- Siapkan presentasi proyek akhir dengan *checklist* di halaman UAS.

Buka bahan pertemuan ini di GitHub:

- **Informasi dan kisi-kisi**
- **Praktik Python**
- **Praktik Java**
- **Latihan soal**
- **Pembahasan**

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-16-UAS

Pindai untuk membuka folder pertemuan ini di GitHub